board(3).py
Code

game(3).py
Code

main(3).py
Code

README(20261011-160020).md
Document

requirements(20261011-160020).txt
Document

rules(3).py
Code

inspect the existing Checkers project, especially board.py, rules.py, and game.py. Don't change anything yet.

I inspected the existing Checkers project **without changing anything**.

Current structure

- `main.py` → only starts the game with `Checkers().run()`. main
- `game.py` → handles the game loop, input, turn switching, and calls the rule functions. game
- `board.py` → creates the 8×8 board and moves pieces from one square to another. board
- `rules.py` → currently contains ordinary movement, capture validation, and promotion. rules
- `README.md` → defines the intended tasks: capture removal, win/no-move detection, forced captures, multi-captures, kings, promotion, and feedback. README

Important current behavior

1. Board representation

Pieces are represented as:

- `R` = Red
- `B` = Black
- `RK` = Red king
- `BK` = Black king
- `.` = empty

The initial board places Black in rows 0–2 and Red in rows 5–7. board

2. Ordinary movement

`simple_move()` currently allows only one diagonal square forward. Red moves upward (−1 row), while Black moves downward (+1). rules

There is **no king-specific movement** in this function yet.

3. Captures

`capture_move()` checks for a two-square diagonal jump and verifies that the middle square contains an opponent piece. rules

However, it **only validates the capture**. It does not remove the jumped piece.

The `game.py` flow simply calls:

```
capture_move(...)
    ↓
move_piece(...)
```

and `move_piece()` only moves the player's piece from start to destination. board

So the README's Task 1 defect is present: **the captured piece currently remains on the board.**

4. Promotion

`promote()` correctly checks the back rows and changes:

- `R` → `RK` on row 0

- `B` → `BK` on row 7 ‹/› rules

But because the movement functions currently don't treat `RK` / `BK` differently, **king movement in both directions is not implemented yet.**

5. Game flow

After every accepted move, `game.py` calls `promote()` and then immediately switches players. ‹/› game

There is currently no visible handling for:

- forced captures
- multiple captures in one turn
- win detection
- no-legal-move detection
- keeping the same player during a multi-capture
- explicit capture removal
- king movement

Overall assessment

The project is currently a **small, clean modular base** rather than something that needs to be rewritten. The existing separation is appropriate:

`board.py` → board/state operations
`rules.py` → movement rules
`game.py` → turn/input/game flow
`main.py` → entry point

The README specifically says to inspect the existing code and **not replace the whole project with an unrelated implementation**, so future changes should build incrementally on this structure. README

I have not modified any of the files.

Fix the current capture bug where the jumped opponent piece is not removed.

Make the change in game.py,

> Give me the exact code to add/change and clearly tell me exactly where to put it. Don't rewrite the rest of the project.

Yes. Since the current `game.py` already detects a capture with `capture_move()` and then calls `move_piece()`, we only need to remove the jumped piece **inside the capture branch**.

Change only `game.py`

Find this existing section around **lines 37–43**:

Python

```
start, end = (sr, sc), (er, ec)
if capture_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)
elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)
else:
    print("Invalid move.")
    continue
```

Replace **only that section** with:

Python

```
start, end = (sr, sc), (er, ec)
if capture_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)

    # Remove the opponent piece that was jumped
    mr = (sr + er) // 2
    mc = (sc + ec) // 2
    self.board[mr][mc] = "."

elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)
else:
    print("Invalid move.")
    continue
```

Exactly what this does

For a capture such as:

```
Plain text
Red:   (5,2)
Black: (4,3)
Red:   (3,4)
```

the middle square is calculated:

```
Python
mr = (sr + er) // 2
mc = (sc + ec) // 2
```

which gives `(4,3)`, and then:

```
Python
self.board[mr][mc] = "."
```

removes the jumped Black piece.

This fits the existing project structure: `game.py` already handles the accepted move, while `board.py` continues to handle the actual piece movement. game

Do not change `board.py` or `rules.py` for this fix.

Add win and no-move detection.

Make the changes in game.py. Add the has_legal_move() method inside the Checkers class, and update run() where the current player's pieces and legal moves are checked.

Give me the exact code and clearly tell me where each part goes. Don't change unrelated code.

Yes. Make **only these changes in `game.py`**.

1. Add `has_legal_move()` inside the `Checkers` class

Put this method **after `print_board()` and before `run()`**.

So you currently have:

Python

```
def print_board(self):
    print("\n   " + " ".join(str(c) for c in range(SIZE)))
    for r, row in enumerate(self.board):
        print(f"{r}  " + " ".join(row))

def run(self):
```

Change it to:

Python

```
def print_board(self):
    print("\n   " + " ".join(str(c) for c in range(SIZE)))
    for r, row in enumerate(self.board):
        print(f"{r}  " + " ".join(row))

def has_legal_move(self, player):
    for r in range(SIZE):
        for c in range(SIZE):
            if self.board[r][c] not in (player, player + "K"):
                continue

            start = (r, c)

            for er in range(SIZE):
                for ec in range(SIZE):
                    end = (er, ec)

                    if capture_move(self.board, player, start, end):
                        return True
```

This checks every piece belonging to the current player and tests every possible destination using the **existing** `capture_move()` and `simple_move()` rules. No changes to `rules.py` are needed.

2. Update `run()` to detect a player with no pieces

Inside `run()`, find this part:

Python

```
        while True:
            self.print_board()
            raw = input(f"{self.player}> ").strip().lower().split()
```

Replace it with:

Python

```
        while True:
            self.print_board()

            if not any(
                piece in (self.player, self.player + "K")
                for row in self.board
                for piece in row
            ):
                winner = "B" if self.player == "R" else "R"
                print(f"{winner} wins! {self.player} has no pieces left.")
                return

            if not self.has_legal_move(self.player):
                winner = "B" if self.player == "R" else "R"
                print(f"{winner} wins! {self.player} has no legal moves.")
                return

            raw = input(f"{self.player}> ").strip().lower().split()
```

What this adds

Before asking the player for a move, the game now checks:

1. **Does the current player have any pieces?**
   - If not, the opponent wins.
2. **Does the current player have any legal move?**
   - If not, the opponent wins.

The rest of your existing `run()` logic remains unchanged. The current game loop already handles input, capture/simple moves, promotion, and switching players. ‹/› game

Final `game.py` structure

You should now have:

‹/› Plain text

```
class Checkers
├── __init__()
├── print_board()
├── has_legal_move()      ← ADD THIS
└── run()                 ← ADD the two checks near the top of while loop
```

Do not change `board.py`, `rules.py`, or `main.py`.

Now add the remaining Checkers rules.

Make the movement and capture rule changes in rules.py, so kings can move and capture in both directions.

Make the turn-handling changes in game.py so captures are forced and the same piece can make multiple captures in one turn.

Keep promotion working correctly.

Give me the exact code for each change and clearly tell me which file, function, and location to put it in. Don't rewrite the project.

Yes. We can do this with **targeted changes only** in `rules.py` and `game.py`.

The changes are:

1. `rules.py` — allow kings to move forward **and backward**.
2. `rules.py` — allow kings to capture forward **and backward**.

3. `game.py` — detect whether **any capture is available** and force a capture when one exists.
4. `game.py` — after a capture, check whether the **same piece can capture again**.
5. `game.py` — keep the same player during a multi-capture.
6. Promotion remains handled by your existing `promote()` function. The existing promotion logic already converts `R → RK` and `B → BK`. ‹/› rules

---

1. Change `rules.py`

Replace `simple_move()`

In **`rules.py`**, find the existing function:

‹/› Python

```
def simple_move(board, player, start, end):
    sr, sc = start
    er, ec = end
    direction = -1 if player == "R" else 1
    return (
        board[er][ec] == "." and
        abs(er - sr) == 1 and abs(ec - sc) == 1 and
        er - sr == direction
    )
```

Replace **only this function** with:

‹/› Python

```
def simple_move(board, player, start, end):
    sr, sc = start
    er, ec = end
    piece = board[sr][sc]

    # Kings can move in both directions
    if piece == player + "K":
        return (
            board[er][ec] == "." and
            abs(er - sr) == 1 and
            abs(ec - sc) == 1
        )
```

```
    # Normal pieces move forward only
    direction = -1 if player == "R" else 1

    return (
        board[er][ec] == "." and
        abs(er - sr) == 1 and
        abs(ec - sc) == 1 and
```

Location

File: `rules.py`

Function: `simple_move()`

Action: Replace the existing function completely.

This keeps normal pieces moving forward while allowing `RK` and `BK` to move diagonally in either direction.

---

2. Change `capture_move()` in `rules.py`

Find the existing:

Python

```
def capture_move(board, player, start, end):
    sr, sc = start
    er, ec = end
    direction = -1 if player == "R" else 1
    mr, mc = (sr + er) // 2, (sc + ec) // 2
    return (
        board[er][ec] == "." and
        abs(er - sr) == 2 and abs(ec - sc) == 2 and
        er - sr == 2 * direction and
        board[mr][mc] not in (".", player)
    )
```

Replace **only this function** with:

Python

```
def capture_move(board, player, start, end):
```

```
    sr, sc = start
    er, ec = end
    piece = board[sr][sc]

    mr, mc = (sr + er) // 2, (sc + ec) // 2

    # Basic two-square diagonal capture requirements
    if board[er][ec] != ".":
        return False

    if abs(er - sr) != 2 or abs(ec - sc) != 2:
        return False

    # The jumped piece must belong to the opponent
    if board[mr][mc] in (".", player, player + "K"):
        return False

    # Kings can capture in both directions
    if piece == player + "K":
```

Location

File: `rules.py`

Function: `capture_move()`

Action: Replace the existing function completely.

Now:

- `R` captures upward.
- `B` captures downward.
- `RK` captures upward or downward.
- `BK` captures upward or downward.

The existing capture logic already identifies the middle square as `(sr + er) // 2, (sc + ec) // 2`; this change preserves that structure while adding king behavior. ‹/› rules

---

3. Add forced-capture checking in `game.py`

You already added `has_legal_move()` to the `Checkers` class.

Now add **one more method immediately after `has_legal_move()` and before `run()`**.

Add:

Python

```
    def has_capture(self, player):
        for r in range(SIZE):
            for c in range(SIZE):
                if self.board[r][c] not in (player, player + "K"):
                    continue

                start = (r, c)

                for er in range(SIZE):
                    for ec in range(SIZE):
                        end = (er, ec)

                        if capture_move(self.board, player, start, end):
                            return True

        return False
```

So your class should now have:

Plain text

```
__init__()
print_board()
has_legal_move()
has_capture()       ← ADD THIS
run()
```

This method answers:

> "Does this player have **any capture anywhere on the board**?"

That is what we need for the forced-capture rule.

---

4. Change the move-handling section in `game.py`

Inside `run()`, find the current section that looks like this:

Python

```
start, end = (sr, sc), (er, ec)
if capture_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)

    # Remove the opponent piece that was jumped
    mr = (sr + er) // 2
    mc = (sc + ec) // 2
    self.board[mr][mc] = "."

elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)
else:
    print("Invalid move.")
    continue
```

Replace **only this block** with:

Python

```
            start, end = (sr, sc), (er, ec)

            # Check whether the player has any capture available
            forced_capture = self.has_capture(self.player)

            if forced_capture and not capture_move(
                self.board, self.player, start, end
            ):
                print("You must capture when a capture is available.")
                continue

            if capture_move(self.board, self.player, start, end):
                move_piece(self.board, start, end)

                # Remove the jumped opponent piece
                mr = (sr + er) // 2
                mc = (sc + ec) // 2
                self.board[mr][mc] = "."

                # Check for another capture with the same piece
```

However, this uses one small helper that we need to add.

5. Add `has_capture_from()` to `game.py`

Put this **directly after `has_capture()` and before `run()`**:

Python

```
def has_capture_from(self, player, start):
    sr, sc = start

    for er in range(SIZE):
        for ec in range(SIZE):
            end = (er, ec)

            if capture_move(self.board, player, start, end):
                return True

    return False
```

So the relevant part of your `Checkers` class becomes:

Python

```
def has_legal_move(self, player):
    for r in range(SIZE):
        for c in range(SIZE):
            if self.board[r][c] not in (player, player + "K"):
                continue

            start = (r, c)

            for er in range(SIZE):
                for ec in range(SIZE):
                    end = (er, ec)

                    if capture_move(self.board, player, start, end):
                        return True

                    if simple_move(self.board, player, start, end):
                        return True

    return False
```

One important correction to the multi-capture code

There is a subtle issue: **promotion should happen immediately when a piece reaches the back row**, and the current turn should then end rather than allowing a newly crowned king to continue capturing.

So inside the multi-capture loop, after:

```
end = next_end
```

add:

```
# Promotion ends the current capture sequence
if (
    self.board[end[0]][end[1]] == "R"
    and end[0] == 0
) or (
    self.board[end[0]][end[1]] == "B"
    and end[0] == SIZE - 1
):
    promote(self.board)
    break
```

Then the `promote(self.board)` after the `while` can remain as a safety check.

Result

After these targeted changes, your rules will work like this:

Normal piece

```
R → moves/captures upward
```

```
B → moves/captures downward
```

King

```
Plain text
RK → moves/captures upward AND downward
BK → moves/captures upward AND downward
```

Forced capture

If **any** opponent piece can be captured, the player cannot make an ordinary move.

Multiple captures

After:

```
Plain text
R jumps B
```

the same `R` piece gets another opportunity:

```
Plain text
R jumps B
```

and continues until no capture is available from that piece.

Turn switching

The player **does not change** during the multi-capture sequence. Once the sequence ends, the normal player switch happens.

Promotion

Your existing `promote()` remains in `rules.py`; it still converts the piece when it reaches the correct back row. rules

Files changed: `rules.py` and `game.py` only.

Files untouched: `board.py` , `main.py` .
No project rewrite or new dependencies.

Add simple terminal feedback for normal moves, captures, promotions, and game-ending moves.

Make these changes only in game.py, any small helper needed there.

Give me the exact code and tell me exactly where to insert or replace it. Keep the messages short and don't change the game rules.

Yes. Since the current `game.py` already contains the rule/turn logic, we only need to add **short `print()` messages**. No changes to `rules.py` , `board.py` , or the game rules.

1. Add a small helper in `game.py`

Inside the `Checkers` class, put this method **after `has_capture_from()` and before `run()`** :

Python

```
    def announce_move(self, player, start, end, capture=False):
        action = "captures" if capture else "moves"
        print(f"{player}: {start} {action} {end}")
```

So the order is:

Plain text

```
__init__()
print_board()
has_legal_move()
has_capture()
has_capture_from()
announce_move()    ← ADD HERE
run()
```

2. Add feedback for a normal move

Inside `run()`, find the normal-move section:

Python

```
elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)
    promote(self.board)
```

Replace it with:

Python

```
elif simple_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)
    self.announce_move(self.player, start, end)
    promote(self.board)
```

You'll now get something like:

Plain text

```
R: (5, 0) moves (4, 1)
```

---

3. Add feedback for captures

In the capture section, find:

Python

```
if capture_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)

    # Remove the jumped opponent piece
    mr = (sr + er) // 2
    mc = (sc + ec) // 2
    self.board[mr][mc] = "."
```

Change it to:

Python

```
if capture_move(self.board, self.player, start, end):
    move_piece(self.board, start, end)

    # Remove the jumped opponent piece
    mr = (sr + er) // 2
    mc = (sc + ec) // 2
    self.board[mr][mc] = "."

    self.announce_move(self.player, start, end, capture=True)
```

This gives:

Plain text

```
R: (5, 0) captures (3, 2)
```

---

4. Add feedback for additional captures

Inside your existing **multiple-capture `while` loop**, find:

Python

```
move_piece(self.board, end, next_end)

# Remove the jumped opponent piece
mr = (end[0] + er) // 2
mc = (end[1] + ec) // 2
self.board[mr][mc] = "."

end = next_end
```

Change it to:

Python

```
move_piece(self.board, end, next_end)

# Remove the jumped opponent piece
mr = (end[0] + er) // 2
mc = (end[1] + ec) // 2
self.board[mr][mc] = "."

self.announce_move(self.player, end, next_end, capture=True)

end = next_end
```

So a multiple capture will show:

Plain text

```
R: (5, 0) captures (3, 2)
R: (3, 2) captures (1, 4)
```

---

5. Add promotion feedback

Your existing code has:

Python

```
promote(self.board)
```

For the normal-move section, change:

Python

```
move_piece(self.board, start, end)
self.announce_move(self.player, start, end)
promote(self.board)
```

to:

Python

```
move_piece(self.board, start, end)
self.announce_move(self.player, start, end)

old_piece = self.board[end[0]][end[1]]
promote(self.board)

if self.board[end[0]][end[1]] != old_piece:
    print(f"{self.player} promotes to king!")
```

Do the same after a capture sequence where promotion is checked.

This produces:

Plain text

```
R: (2, 5) moves (1, 6)
R promotes to king!
```

The existing `promote()` function remains unchanged. rules

---

6. Add game-ending feedback

You previously added these checks near the beginning of the `while True` loop:

Python

```
if not any(
    piece in (self.player, self.player + "K")
    for row in self.board
    for piece in row
):
    winner = "B" if self.player == "R" else "R"
    print(f"{winner} wins! {self.player} has no pieces left.")
    return

if not self.has_legal_move(self.player):
    winner = "B" if self.player == "R" else "R"
    print(f"{winner} wins! {self.player} has no legal moves.")
    return
```

Keep these exactly as they are.

They already provide the game-ending feedback:

```
Plain text
B wins! R has no pieces left.
```

or:

```
Plain text
B wins! R has no legal moves.
```

The no-move/piece checks are part of the existing `game.py` turn flow. game

---

Final changes

Only `game.py` needs modification:

Feedback	Change
Normal move	`announce_move(..., capture=False)`
Capture	`announce_move(..., capture=True)`
Multiple capture	Announce each jump
Promotion	Print `"R promotes to king!"`
Win/no moves	Keep existing win messages

Do not modify the movement rules, capture rules, promotion rules, or turn logic.

Do a final review of the current Checkers project against the README.

If everything is correct, don't change anything. If something is missing or incorrect, give me the exact code needed and clearly state the file, function, and exact place where I should make the change.

Keep the existing project structure and don't rewrite the project.

I reviewed the **currently attached project files** against the README requirements. The important point is that the attached `game.py` is still the original version; the changes we discussed in the previous messages are **not present in the attached file**.

So, based strictly on the files currently provided, the project is **not yet complete**. The README requires Tasks 1–4, including capture removal, win/no-move detection, forced captures, multi-captures, kings, promotion, and feedback. README

What is currently missing

README requirement	Current status
Capture removes jumped piece	❌ Missing
Win when opponent has no pieces	❌ Missing
Win when opponent has no legal moves	❌ Missing
Forced captures	❌ Missing
Multiple captures	❌ Missing
King movement both directions	❌ Missing
King captures both directions	❌ Missing
Promotion	✅ Present
Move/capture/promotion feedback	❌ Missing
Invalid coordinates	✅ Present
Quit	✅ Present

The current `game.py` simply performs one capture/simple move, promotes, and immediately switches players. ‹/› game

The current `rules.py` only allows normal pieces to move/capture in one direction. ‹/› rules

However — don't make changes yet

There is one important issue with doing another patch based on this review:

The attached files are the original versions, not the modified versions from our previous steps.

For example, the attached `game.py` does **not** contain the `has_legal_move()`, `has_capture()`, or `has_capture_from()` methods we added in our previous instructions. ‹/› game

Therefore, I don't want to give you another set of patches that could accidentally conflict with the code you have actually modified locally.

What you should do

If you have already applied the previous changes to your local `game.py` and `rules.py`, **upload those two current files**.

Then I can do a true final review of the **actual final code** against every README requirement and tell you either:

> **Everything is correct — make no changes.**

or give you only the specific missing/incorrect changes, with the exact file, function, and insertion point.

The README itself explicitly requires testing ordinary moves, captures, multiple captures, forced captures, promotion, king movement, no-move positions, invalid coordinates, and quitting.

README